SRI CHANDRASEKHARENDRA SARASWATHIVISWA MAHAVIDYALAYA
(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Lab Manual
Operating System

Prepared by

Dr.M.Gayathri,

Assistant Professor (Stage-II)

Index

1. DEPARTMENT VISION

The Computer Science Engineering Department aims at providing accessible and relevant programs that are directly applicable to the dynamic environment in which our students will work, live and contribute.

2. DEPARTMENT MISSION

To produce high profile future computer experts, endowed with high quality knowledge of automation and other allied sciences, sufficiently capable of proving their professional mettle in the present-day competitive world with confidence.

3. PROGRAMME EDUCATIONAL OBJECTIVES (PEOs)

- Provide engineering insight to problem solving to succeed in Technical Profession through precise education and to prepare students to excel in postgraduate programs.
- To provide students with fundamental knowledge and ability to expertise in Computer Science and Engineering.
- Prepare students with good scientific and engineering breadth so as to analyze, design and create products, solutions to problems in the area of Computer Science and Engineering.
- To inculcate in students professional, effective communication skills, team work, multidisciplinary approach and an ability to relate engineering issues to broader social context.
- Prepare students to be aware of excellence, leadership, written ethical codes and guidelines and lifelong learning needed for successful professional career by providing them with an excellent academic environment.

4. PROGRAM OUTCOME(S) (POs)

i) Engineering Knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

ii) Problem Analysis: Identify, formulate, review research literature, and analyzes complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences and engineering sciences.

iii) Design/Development of Solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

iv) Conduct Investigations of Complex Problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions. Department of Computer Science and Engineering Page 3 of 179

v) Modern Tool Usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

vi) The Engineer and Society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

vii) Environment and Sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

viii) Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

ix) Individual **and Team Work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

x) Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

xi) Project Management and Finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

Xii) Life-long Learning: Recognize the need for, and have the preparation and ability to engage in independent and lifelong learning in the broadest context of technological change.

5. PO – CO/PSO Mapping:

	PO 01	PO 02	PO 03	PO 04	PO 05	PO 06	PO 07	PO 08	PO 09	PO 10	PO 11	PO 12	PSO 01	PSO 02	PSO 03
CO 01	S												M		
CO 02		S	M											S	
CO 03			S	M										S	
CO 04				S										M	
CO 05					S								S		M

6. PRE-REQUISITES

Basic Knowledge in any programming knowledge.

7. OBJECTIVES

- Various Linux operating systems commands
- Shell Programming
- Implement various CPU scheduling algorithm
- Implement various memory allocation methods.
- Installation of Guest OS

8. COURSE OUTCOMES

At the end of the course the student will be able to:

CO1: Understand the various Linux commands.

CO2: Simulation of CPU Scheduling Algorithms. (FCFS, RR, SJF).

CO3: Simulation of Banker's Algorithm for Deadlock Avoidance.

CO4: Implement various memory allocation methods.

CO5: Install a Guest OS in Virtual Box / VMware.

9. SYSTEM REQUIREMENTS:

1. Hardware Requirements

a. Intel / AMD based system with minimum Dual Core Processor

b. 4 GB RAM or higher

c. Minimum 20 GB free hard disk space

d. Standard keyboard, mouse, and monitor

2. Software Requirements

a. Operating System: Linux (Ubuntu 20.04 / Fedora or higher)

b. Programming Language: C, Shell Scripting (Bash)

c. Compiler: GCC Compiler

d. Shell: Bash

e. Utilities / Editors: Terminal, VI / Nano / Gedit

10. ABOUT PROGRAM FOR OPERATING SYSTEM LAB:

Operating Systems Laboratory provides practical exposure to the fundamental concepts of operating systems through hands-on experiments. The laboratory enables students to understand process management, CPU scheduling, inter-process communication, synchronization, memory management, and disk scheduling by implementing standard algorithms and system calls in a Linux environment. Through this lab, students gain practical skills in using UNIX/Linux commands, shell scripting, and C programming to analyze and simulate core operating system functionalities, thereby strengthening their understanding of theoretical concepts and preparing them for real-world system-level programming.

11. GUIDELINES TO STUDENTS:

The following are the guidelines for the students in writing the observation, maintaining the record and Do's and Don'ts in the Laboratory

A. Writing of the experiment in the Observation Book

The students will write the today's experiment in the Observation book as per the following format: a) Name of the experiment b) Aim c) Writing the program d) Viva-Voce Questions and Answers e) Errors observed (if any) during compilation/execution f) Signature of the Faculty.

B. Guide Lines to Students in Lab Disciplinary to be maintained by the students in the Lab

- Students are required to carry their lab observation book and record book with completed experiments while entering the lab.
- Students must use the equipment with care. Any damage is caused student is punishable
- Students are not allowed to use their cell phones/pen drives/ CDs in labs.
- Students need to be maintain proper dress code along with ID Card
- Students are supposed to occupy the computers allotted to them and are not supposed to talk or make noise in the lab. Students, after completion of each experiment they need to be updated in observation notes and same to be updated in the record.
- Lab records need to be submitted after completion of experiment and get it corrected with the concerned lab faculty.

- If a student is absent for any lab, they need to be completed the same experiment in the free time before attending next lab.

C. Steps to perform experiments in the lab by the student

Step 1: Students have to write the date, aim, and Software & Hardware requirements for that experiment in the observation book.

Step 2: Students have to listen and understand the experiment explained by the faculty and note down the important points in the observation book.

Step 3: Students need to write procedure/algorithm in the observation book.

Step 4: Analyse and Develop/implement the logic of the program by the student in respective platform

Step 5: After approval of logic of the experiment by the faculty then the experiment has to be executed on the system.

Step 6: After successful execution the results are to be shown to the faculty and noted the same in the observation book.

Step 7: Students need to attend the Viva-Voce on that experiment and write the same in the observation book.

Step 8: Update the completed experiment in the record and submit to the concerned faculty in-charge.

D. Instructions to maintain the record

- Before start of the first lab they have to buy the record and bring the record to the lab.
- Regularly (Weekly) update the record after completion of the experiment and get it corrected with concerned lab in-charge for continuous evaluation.
- In case the record is lost inform the same day to the faculty in charge and get the new record within 2 days the record has to be submitted and get it corrected by the faculty.
- If record is not submitted in time or record is not written properly, the evaluation marks (5M) will be deducted.

E. General laboratory instructions

1. Students are advised to come to the laboratory at least 5 minutes before (to the starting time), those who come after 5 minutes will not be allowed into the lab.
2. Plan your task properly much before to the commencement, come prepared to the lab with the synopsis / program / experiment details.
3. Student should enter into the laboratory with: a. Laboratory observation notes with all the details (Problem statement, Aim, Algorithm, Procedure, Program, Expected Output, etc.,) filled in for the lab session. b. Laboratory Record updated up to the

last session experiments and other utensils (if any) needed in the lab. c. Proper Dress code and Identity card.

4. Sign in the laboratory login register, write the TIME-IN, and occupy the computer system allotted to you by the faculty.
5. Execute your task in the laboratory, and record the results / output in the lab observation note book, and get certified by the concerned faculty.
6. All the students should be polite and cooperative with the laboratory staff, must maintain the discipline and decency in the laboratory.
7. Computer labs are established with sophisticated and high end branded systems, which should be utilized properly.
8. Students must keep their mobile phones in SWITCHED OFF mode during the lab sessions. Misuse of the equipment, misbehaviours with the staff and systems etc., will attract severe punishment.
9. Students must take the permission of the faculty in case of any urgency to go out; if anybody found loitering outside the lab / class without permission during working hours will be treated seriously and punished appropriately.
10. Students should LOG OFF/ SHUT DOWN the computer system before he/she leaves the lab after completing the task (experiment) in all aspects. He/she must ensure the system / seat is kept properly.

LIST OF PROGRAMS

	b. Worst Fit c. Best Fit	
10	Install any guest operating system like Linux using VMware.	
CONTENT BEYOND SYLLABUS		
1	Multithreaded Programming using POSIX Threads	
2	Real-Time CPU Scheduling Algorithms	

REFERENCES

1. https://sites.google.com/view/oslaboratory/home?
2. https://www.cse.iitb.ac.in/~mythili/os/?

Ex: No: 1 a	Installation of Windows Operating System
Date:	

AIM:

To install the Windows Operating System on a personal computer and understand the basic steps involved in operating system installation, including disk partitioning, file system selection, and system configuration.

PROCEDURE: INSTALLATION OF WINDOWS OPERATING SYSTEM

1. Obtain a bootable Windows installation media (USB/DVD) with a valid Windows ISO file.
2. Insert the bootable media into the system and restart the computer.
3. Enter the BIOS/UEFI setup and set the boot priority to USB/DVD.
4. Save the BIOS settings and reboot the system.
5. When the Windows setup screen appears, select the language, time, and keyboard settings, and click Next.
6. Click Install Now and enter the product key if prompted, or select Skip.
7. Choose the appropriate Windows edition and accept the license agreement.
8. Select Custom Installation to install Windows on a specific partition.
9. Create, delete, or format disk partitions as required and select the target partition.
10. Allow the system to copy Windows files and complete the installation automatically.
11. After installation, configure basic settings such as username, password, date, time, and network.
12. Once the setup is completed, the Windows desktop will be displayed, indicating successful installation

RESULT:

Thus, the Windows Operating System was successfully installed on the system and the basic system configuration was completed as per the given procedure.

Ex: No: 2	Basic Linux Commands
Date:	

AIM:

To understand and practice basic Linux commands used for file and directory management, process and system status monitoring, text file operations, and file permission handling in a Linux operating system environment.

DESCRIPTION:

Linux provides a powerful command-line interface that allows users to interact with the operating system efficiently. In this experiment, commonly used Linux commands are practiced and categorized into file and directory related commands, process and status information commands, text-related commands, and file permission commands. These commands help users manage files, monitor system performance, manipulate text files, and control access permissions.

a. File and Directory Related Commands

These commands are used to navigate the file system and manage files and directories.

1. pwd (Print Working Directory)

Displays the current working directory.

pwd

2. ls (List)

Lists files and directories.

ls
ls -l
ls -a

3. cd (Change Directory)

Changes the current directory.

cd Documents

4. mkdir (Make Directory)

Creates a new directory.

mkdir oslab

5. mkdir -p

Creates parent and subdirectories.

mkdir -p cse/os/lab

6. rmdir

Deletes an empty directory.

rmdir oslab

7. touch

Creates an empty file.

touch file.txt

8. cp (Copy)

Copies files or directories.

cp file1.txt file2.txt
cp -r dir1 dir2

9. mv (Move / Rename)

Moves or renames files.

mv old.txt new.txt

10. rm (Remove)

Deletes files or directories.

rm file.txt
rm -r folder

11. file

Identifies file type.

file file.txt

12. stat

Displays detailed file information.

stat file.txt

b. Process and Status Information Commands

These commands help monitor processes and system performance.

1. ps

Displays running processes.

ps
ps -ef

2. top

Shows real-time process and CPU usage.

top

3. htop

Enhanced interactive process viewer.

htop

4. kill

Terminates a process using PID.

kill 1234

5. pkill

Terminates process by name.

pkill firefox

6. jobs

Displays background jobs.

jobs

7. fg

Brings job to foreground.

fg

8. bg

Resumes job in background.

bg

9. uptime

Displays system running time.

uptime

10. who

Shows logged-in users.

who

11. whoami

Displays current user.

whoami

12. free

Displays memory usage.

free -h

13. df

Shows disk space usage.

df -h

14. du

Shows directory size.

du -sh folder

c. Text Related Commands

These commands are used to view, search, and process text files.

1. cat

Displays file contents.

cat file.txt

2. tac

Displays file in reverse order.

tac file.txt

3. more

Displays file page by page.

more file.txt

4. less

Scrollable file viewing.

less file.txt

5. head

Displays first lines of file.

head file.txt

6. tail

Displays last lines of file.

tail file.txt

7. grep

Searches text pattern.

grep "Linux" file.txt

8. wc

Counts lines, words, and characters.

wc file.txt

9. sort

Sorts text data.

sort names.txt

10. uniq

Removes duplicate lines.

uniq file.txt

11. cut

Extracts columns.

cut -d':' -f1 /etc/passwd

12. paste

Merges files line by line.

paste file1 file2

13. tr

Translates characters.

tr a-z A-Z < file.txt

14. vi / vim

Text editor.

vi file.txt

15. nano

Simple text editor.

nano file.txt

d. File Permission Commands

These commands control access to files and directories.

1. ls -l

Displays file permissions.

ls -l

2. chmod

Changes file permissions.

chmod 755 file.sh
chmod u+x file.sh

3. chown

Changes file owner.

chown user file.txt

4. chgrp

Changes group ownership.

chgrp staff file.txt

5. umask

Displays default permission settings.

umask

Permission Format Example

-rwxr-xr--

- r – Read
- w – Write
- x – Execute

Result:

Thus, Linux commands related to file and directory management, process monitoring, text processing, and file permission handling were successfully practiced and executed.

Ex: No: 3a	Execute shell program to find the whether a Number is even or odd
Date:	

AIM:

To write a shell program to check whether a given number is even or odd.

ALGORITHM:

1. Read a number from the user.
2. Find the remainder when the number is divided by 2.
3. If the remainder is 0, display “Even”.
4. Otherwise, display “Odd”.

PROGRAM:

```
#!/bin/bash
echo "Enter a number:"
read n
if [ $((n % 2)) -eq 0 ]
then
  echo "The number is Even"
else
  echo "The number is Odd"
fi
```

EXECUTION PROCEDURE:

1. Open the terminal.
2. Create a file using vi evenodd.sh.
3. Type the program and save the file.
4. Provide execute permission using chmod +x evenodd.sh.
5. Execute the program using ./evenodd.sh.

SAMPLE OUTPUT:

Enter a number:

6

The number is Even

RESULT:

Thus, the shell program to check whether a number is even or odd was executed successfully.

Ex: No: 3b	Shell Program to Find the Biggest of Two Numbers
Date:	

AIM:

To write a shell program to find the biggest of two given numbers.

ALGORITHM:

1. Read two numbers from the user.
2. Compare the two numbers.
3. Display the larger number.

PROGRAM:

```
#!/bin/bash
echo "Enter first number:"
read a
echo "Enter second number:"
read b
if [ $a -gt $b ]
then
  echo "Biggest number is $a"
else
  echo "Biggest number is $b"
fi
```

EXECUTION PROCEDURE:

1. Create a file big2.sh.
2. Enter the program and save it.
3. Give execution permission using chmod +x big2.sh.
4. Run the program using ./big2.sh.

SAMPLE OUTPUT

Enter first number:

10

Enter second number:

25

Biggest number is 25

RESULT:

Thus, the shell program to find the biggest of two numbers was executed successfully.

Ex: No: 3c	Shell Program to Find the Biggest of Three Numbers
Date:	

AIM:

To write a shell program to find the biggest of three given numbers.

ALGORITHM:

1. Read three numbers from the user.
2. Compare the numbers using conditional statements.
3. Display the largest number.

PROGRAM:

```
#!/bin/bash
echo "Enter three numbers:"
read a
read b
read c

if [ $a -gt $b ] && [ $a -gt $c ]
then
  echo "Biggest number is $a"
elif [ $b -gt $c ]
then
  echo "Biggest number is $b"
else
  echo "Biggest number is $c"
fi
```

EXECUTION PROCEDURE:

1. Create a file big3.sh.
2. Type the program and save it.
3. Provide execute permission.
4. Execute using ./big3.sh

SAMPLE OUTPUT:

Enter three numbers:

12

45

30

Biggest number is 45

RESULT:

Thus, the shell program to find the biggest of three numbers was executed successfully.

Ex: No: 3d	Shell Program to Find the Factorial of a Number
Date:	

AIM:

To write a shell program to find the factorial of a given number.

ALGORITHM:

1. Read a number from the user.
2. Initialize factorial value to 1.
3. Use a loop to multiply numbers from 1 to the given number.
4. Display the factorial.

PROGRAM:

```
#!/bin/bash
echo "Enter a number:"
read n
fact=1
for (( i=1; i<=n; i++ ))
do
  fact=$((fact * i))
done
echo "Factorial of $n is $fact"
```

EXECUTION PROCEDURE:

1. Create a file factorial.sh.
2. Enter and save the program.
3. Provide execute permission.
4. Run the program.

SAMPLE OUTPUT:

Enter a number:

5

Factorial of 5 is 120

RESULT:

Thus, the shell program to find the factorial of a number was executed successfully.

Ex: No: 3e	Shell Program to Display Fibonacci Series
Date:	

AIM:

To write a shell program to display the Fibonacci series up to a given number of terms.

ALGORITHM:

1. Read the number of terms.
2. Initialize first two values as 0 and 1.
3. Display the first two terms.
4. Use a loop to generate remaining terms.
5. Display the Fibonacci series.

PROGRAM

```
#!/bin/bash
echo "Enter number of terms:"
read n
a=0
b=1
echo "Fibonacci Series:"
echo $a
echo $b
for (( i=2; i<n; i++ ))
do
  c=$((a + b))
  echo $c
  a=$b
  b=$c
done
```

EXECUTION PROCEDURE:

1. Create a file fibonacci.sh.
2. Type the program and save it.
3. Provide execute permission.
4. Execute the program.

SAMPLE OUTPUT:

Enter number of terms:

5

Fibonacci Series:

0

1

1

2

3

RESULT:

Thus, the shell program to generate the Fibonacci series was executed successfully.

Ex: No: 4	Linux Pipes and Filters commands
Date:	

AIM:

To study and practice Linux pipes and filter commands such as grep, awk, and sed for processing and manipulating text data in the Linux environment.

DESCRIPTION:

Linux provides powerful text processing utilities known as filters, which take input, process it, and produce output. These filters can be combined using pipes (|) to perform complex operations efficiently. In this experiment, commonly used filter commands—grep, awk, and sed—are practiced along with pipes to search, extract, and modify text data.

a. Linux Pipes (|)

Description

A pipe is used to pass the output of one command as input to another command.

Syntax

command1 | command2

Example

ls -l | grep ".txt"

Explanation:

Lists only .txt files from the directory.

b. grep Command

Description

grep is used to search for a specific pattern or word in a file.

Syntax

grep [options] pattern filename

Common Options

- -i : Case insensitive search
- -r : Recursive search
- -n : Display line numbers

Examples

grep "Linux" file.txt

Searches for the word *Linux* in file.txt.

grep -i "os" file.txt

Searches for *os* ignoring case.

ps -ef | grep root

Displays processes owned by the root user using pipe.

c. awk Command

Description

awk is a powerful pattern scanning and processing language used to extract and manipulate data from files.

Syntax

awk 'pattern {action}' filename

Examples

awk '{print $1}' file.txt

Prints the first column of the file.

awk '{print $1, $3}' file.txt

Prints first and third columns.

df -h | awk '{print $1, $5}'

Displays file system name and disk usage using pipe.

d. sed Command

Description

sed (Stream Editor) is used to perform text transformations such as substitution, deletion, and insertion.

Syntax

sed 'command' filename

Examples

sed 's/Linux/UNIX/' file.txt

Replaces the first occurrence of Linux with UNIX in each line.

sed 's/Linux/UNIX/g' file.txt

Replaces all occurrences of Linux with UNIX.

sed '2d' file.txt

Deletes the second line of the file.

ls | sed 's/.txt/.doc/'

Replaces .txt with .doc in output using pipe.

e. Combined Use of Pipes and Filters

Examples

cat file.txt | grep "OS"

Searches for lines containing *OS*.

ps -ef | grep root | awk '{print $2}'

Displays process IDs of root processes.

cat file.txt | sed 's/error/ERROR/g'

Replaces lowercase error with uppercase ERROR.

Result

Thus, Linux pipes and filter commands such as grep, awk, and sed were successfully executed to process and manipulate text data.

Ex: No: 5	Problems in Decision making statements - Find Even or odd
Date:	

AIM:

To write and execute C programs using various UNIX/Linux system calls to understand file handling, process management, and directory operations.

a) Program using stat() **System Call**

Aim

To obtain file information using the stat() system call.

Algorithm

1. Read file name from the user.
2. Use stat() to retrieve file details.
3. Display file size, inode number, and permissions.

Source Code

```
#include <stdio.h>
#include <sys/stat.h>
int main() {
   struct stat st;
   char fname[50];
   printf("Enter file name: ");
   scanf("%s", fname);
   stat(fname, &st);
   printf("File size: %ld bytes\n", st.st_size);
   printf("Inode number: %ld\n", st.st_ino);
   printf("Permissions: %o\n", st.st_mode);
   return 0;
}
```

Sample Output

Enter file name: test.txt

File size: 120 bytes

Inode number: 256874

Permissions: 100644

Result

Thus, file information was successfully obtained using stat() system call.

b) Program using wait() **System Call**

Aim

To demonstrate parent–child synchronization using wait().

Algorithm

1. Create a child process using fork().
2. Parent waits until child completes execution.
3. Display process termination message.

Source Code

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
int main() {
   pid_t pid = fork();
   if (pid == 0) {
      printf("Child process executing\n");
   } else {
      wait(NULL);
      printf("Parent process resumes after child\n");
   }
   return 0;
}
```

Result

Thus, the parent process waited successfully for the child using wait().

c) Program using getpid() **System Call**

Aim

To display the process ID using getpid().

Source Code

```
#include <stdio.h>
#include <unistd.h>
int main() {
   printf("Process ID: %d\n", getpid());
   return 0;
}
```

Sample Output

Process ID: 3456

Result

Thus, the process ID was displayed using getpid().

d) Program using opendir() **and** readdir()

Aim

To list files in a directory using directory system calls.

Algorithm

1. Open directory using opendir().
2. Read entries using readdir().
3. Display file names.

Source Code

```
#include <stdio.h>
#include <dirent.h>
int main() {
    DIR *dir;
    struct dirent *entry;
    dir = opendir(".");
    while ((entry = readdir(dir)) != NULL) {
        printf("%s\n", entry->d_name);
    }
    closedir(dir);
    return 0;
}
```

Result

Thus, directory contents were listed using opendir() and readdir().

e) Program using open()**,** read()**, and** write()

Aim

To perform file operations using low-level system calls.

Algorithm

1. Open a file using open().
2. Read contents using read().
3. Write contents to output using write().

Source Code

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
int main() {
   int fd;
   char buffer[100];
   fd = open("test.txt", O_RDONLY);
   read(fd, buffer, 100);
   write(1, buffer, 100);
   close(fd);
   return 0;
}
```

Sample Output

(File contents of test.txt displayed)

Result

Thus, file operations were successfully performed using open(), read(), and write() system calls.

Ex: No: 6	Implement Process Management
Date:	

AIM:

To implement process management in Linux using the system calls fork() and exec() and to understand process creation and program execution.

a) Program using fork() **System Call**

Aim

To create a child process using the fork() system call and observe parent and child execution.

Algorithm

1. Create a new process using fork().
2. Check the return value of fork().
3. If return value is 0, it is the child process.
4. If return value is positive, it is the parent process.
5. Display process IDs.

Source Code

```
#include <stdio.h>
#include <unistd.h>
int main() {
   pid_t pid;
   pid = fork();
   if (pid == 0) {
      printf("Child Process\n");
      printf("Child PID: %d\n", getpid());
   } else {
      printf("Parent Process\n");
      printf("Parent PID: %d\n", getpid());
   }
   return 0;
}
```

Execution Procedure

1. Open terminal.
2. Create file using vi fork.c.
3. Compile using gcc fork.c.
4. Execute using. /a.out.

Sample Output

Parent Process

Parent PID: 3450

Child Process

Child PID: 3451

Result

Thus, a child process was successfully created using the fork() system call.

b) Program using exec() **System Call**

Aim

To replace the current process image with a new process using the exec() system call.

Algorithm

1. Create a child process using fork().
2. Use exec() in the child process to execute a new program.
3. Parent process waits for the child.

Source Code

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main() {
   pid_t pid;
   pid = fork();
   if (pid == 0) {
      execl("/bin/ls", "ls", NULL);
      printf("Exec failed\n");
   } else {
      wait(NULL);
      printf("Parent process completed\n");
   }
   return 0;
}
```

Execution Procedure

1. Create file using vi exec.c.
2. Compile using gcc exec.c.
3. Execute using ./a.out.

Sample Output

file1.txt

file2.txt

exec.c

Parent process completed

Result

Thus, a new program was successfully executed using the exec() system call.

Ex: No:7	Implement Various CPU Scheduling Algorithms
Date:	

AIM

To implement and analyze different CPU scheduling algorithms such as FIFO (FCFS), Round Robin, and Shortest Job First (SJF) using C programming.

a) FIFO (FCFS) Scheduling Algorithm

Aim

To implement the First Come First Serve (FCFS) CPU scheduling algorithm.

Algorithm

1. Read number of processes.
2. Read burst time for each process.
3. Calculate waiting time for each process.
4. Calculate turnaround time.
5. Display waiting time and turnaround time.

Source Code

```
#include <stdio.h>
int main() {
   int n, i;
   int bt[10], wt[10], tat[10];
   printf("Enter number of processes: ");
   scanf("%d", &n);
   printf("Enter burst times:\n");
   for (i = 0; i < n; i++)
      scanf("%d", &bt[i]);
   wt[0] = 0;
   for (i = 1; i < n; i++)
      wt[i] = wt[i-1] + bt[i-1];
   for (i = 0; i < n; i++)
      tat[i] = wt[i] + bt[i];
   printf("\nProcess\tBT\tWT\tTAT\n");
   for (i = 0; i < n; i++)
      printf("P%d\t%d\t%d\t%d\n", i+1, bt[i], wt[i], tat[i]);
```

```
    return 0;
}
```

Sample Input

Enter number of processes: 3

Enter burst times:

5 3 8

Sample Output

Process BT WT TAT

P1 5 0 5

P2 3 5 8

P3 8 8 16

Result

Thus, FCFS scheduling algorithm was implemented successfully.

b) Round Robin Scheduling Algorithm

Aim

To implement the Round Robin CPU scheduling algorithm.

Algorithm

1. Read number of processes and burst time.
2. Read time quantum.
3. Execute each process for a fixed time slice.
4. Calculate waiting time and turnaround time.
5. Display results.

Source Code

```
#include <stdio.h>
int main() {
   int n, tq, i, time = 0;
   int bt[10], rt[10], wt[10] = {0}, tat[10];
   printf("Enter number of processes: ");
   scanf("%d", &n);
   printf("Enter burst times:\n");
   for (i = 0; i < n; i++) {
      scanf("%d", &bt[i]);
      rt[i] = bt[i];
   }
   printf("Enter time quantum: ");
   scanf("%d", &tq);
   while (1) {
      int done = 1;
      for (i = 0; i < n; i++) {
         if (rt[i] > 0) {
            done = 0;
            if (rt[i] > tq) {
               time += tq;
               rt[i] -= tq;
            } else {
```

```
                time += rt[i];
                wt[i] = time - bt[i];
                rt[i] = 0;
            }
        }
    }
    if (done)
        break;
}
for (i = 0; i < n; i++)
    tat[i] = wt[i] + bt[i];
printf("\nProcess\tBT\tWT\tTAT\n");
for (i = 0; i < n; i++)
    printf("P%d\t%d\t%d\t%d\n", i+1, bt[i], wt[i], tat[i]);
return 0;
}
```

Sample Input

Enter number of processes: 3

Enter burst times:

10 5 8

Enter time quantum: 2

Sample Output

Process BT WT TAT

P1 10 13 23

P2 5 8 13

P3 8 10 18

Result

Thus, Round Robin scheduling algorithm was implemented successfully.

c) Shortest Job First (SJF – Non-Preemptive)

Aim

To implement the Shortest Job First CPU scheduling algorithm.

Algorithm

1. Read number of processes.
2. Read burst times.
3. Sort processes based on burst time.
4. Calculate waiting time and turnaround time.
5. Display results.

Source Code

```
#include <stdio.h>
int main() {
   int n, i, j;
   int bt[10], wt[10], tat[10], temp;
   printf("Enter number of processes: ");
   scanf("%d", &n);
   printf("Enter burst times:\n");
   for (i = 0; i < n; i++)
      scanf("%d", &bt[i]);
   for (i = 0; i < n-1; i++) {
      for (j = i+1; j < n; j++) {
         if (bt[i] > bt[j]) {
            temp = bt[i];
            bt[i] = bt[j];
            bt[j] = temp;
         }
      }
   }
   wt[0] = 0;
   for (i = 1; i < n; i++)
      wt[i] = wt[i-1] + bt[i-1];
```

```
    for (i = 0; i < n; i++)
        tat[i] = wt[i] + bt[i];
    printf("\nProcess\tBT\tWT\tTAT\n");
    for (i = 0; i < n; i++)
        printf("P%d\t%d\t%d\t%d\n", i+1, bt[i], wt[i], tat[i]);
    return 0;
}
```

Sample Input

Enter number of processes: 3

Enter burst times:

6 2 8

Sample Output

Process BT WT TAT

P1 2 0 2

P2 6 2 8

P3 8 8 16

Result

Thus, the Shortest Job First scheduling algorithm was implemented successfully.

Ex: No: 8	Deadlock Avoidance using Banker's Algorithm
Date:	

AIM:

To write and execute a C program to avoid deadlock using Banker's Algorithm and to determine whether the system is in a safe state.

DESCRIPTION

Banker's Algorithm is a deadlock avoidance algorithm that checks for safe resource allocation. It ensures that the system allocates resources only if the allocation leads to a safe state, thereby preventing deadlock.

ALGORITHM

1. Read number of processes and number of resources.
2. Read allocation matrix.
3. Read maximum requirement matrix.
4. Read available resources.
5. Calculate the need matrix.
6. Find a safe sequence of execution.
7. Display whether the system is in a safe state.

SOURCE CODE

```
#include <stdio.h>
int main() {
   int n, m, i, j, k;
   int alloc[10][10], max[10][10], need[10][10];
   int avail[10], finish[10] = {0}, safeSeq[10];
   printf("Enter number of processes: ");
   scanf("%d", &n);

   printf("Enter number of resources: ");
   scanf("%d", &m);
   printf("Enter allocation matrix:\n");
   for (i = 0; i < n; i++)
```

```
    for (j = 0; j < m; j++)
        scanf("%d", &alloc[i][j]);
printf("Enter maximum matrix:\n");
for (i = 0; i < n; i++)
    for (j = 0; j < m; j++)
        scanf("%d", &max[i][j]);
printf("Enter available resources:\n");
for (i = 0; i < m; i++)
    scanf("%d", &avail[i]);
for (i = 0; i < n; i++)
    for (j = 0; j < m; j++)
        need[i][j] = max[i][j] - alloc[i][j];
for (k = 0; k < n; k++) {
    for (i = 0; i < n; i++) {
        if (!finish[i]) {
            int flag = 1;
            for (j = 0; j < m; j++) {
                if (need[i][j] > avail[j]) {
                    flag = 0;
                    break;
                }
            }
            if (flag) {
                for (j = 0; j < m; j++)
                    avail[j] += alloc[i][j];
                safeSeq[k] = i;
                finish[i] = 1;
            }
        }
    }
}
int safe = 1;
for (i = 0; i < n; i++) {
    if (!finish[i]) {
        safe = 0;
```

```
            break;
        }
    }
    if (safe) {
        printf("\nSystem is in SAFE state.\nSafe sequence: ");
        for (i = 0; i < n; i++)
            printf("P%d ", safeSeq[i]);
    } else {
        printf("\nSystem is in DEADLOCK state.\n");
    }
    return 0;
}
```

EXECUTION PROCEDURE

1. Open terminal.
2. Create a file using vi banker.c.
3. Compile the program using gcc banker.c.
4. Execute the program using ./a.out.
5. Enter the required input values.

SAMPLE INPUT

Enter number of processes: 5

Enter number of resources: 3

Allocation Matrix:

0 1 0

2 0 0

3 0 2

2 1 1

0 0 2

Maximum Matrix:

7 5 3

3 2 2

9 0 2

2 2 2

4 3 3

Available Resources:

3 3 2

SAMPLE OUTPUT

System is in SAFE state.

Safe sequence: P1 P3 P4 P0 P2

RESULT

Thus, deadlock was successfully avoided using Banker's Algorithm and the system was found to be in a safe state.

Ex: No: 9	Memory Allocation Methods
Date:	

AIM:

To write and execute C programs to implement memory allocation techniques such as First Fit, Best Fit, and Worst Fit and to analyze their behavior.

DESCRIPTION

Memory allocation methods are used by the operating system to allocate memory blocks to processes efficiently. This experiment demonstrates how different allocation strategies select memory blocks based on size and availability.

a) FIRST FIT MEMORY ALLOCATION

AIM

To allocate memory using the First Fit strategy.

ALGORITHM

1. Read number of memory blocks and their sizes.
2. Read number of processes and their memory requirements.
3. Allocate the first block that is large enough.
4. Display allocation results.

SOURCE CODE

```
#include <stdio.h>
int main() {
   int nb, np, i, j;
   int block[10], process[10], alloc[10];
   printf("Enter number of blocks: ");
   scanf("%d", &nb);
   printf("Enter block sizes:\n");
   for (i = 0; i < nb; i++)
      scanf("%d", &block[i]);
   printf("Enter number of processes: ");
```

```
    scanf("%d", &np);
    printf("Enter process sizes:\n");
    for (i = 0; i < np; i++) {
        scanf("%d", &process[i]);
        alloc[i] = -1;
    }
    for (i = 0; i < np; i++) {
        for (j = 0; j < nb; j++) {
            if (block[j] >= process[i]) {
                alloc[i] = j;
                block[j] -= process[i];
                break;
            }
        }
    }
    printf("\nProcess\tSize\tBlock\n");
    for (i = 0; i < np; i++) {
        printf("P%d\t%d\t", i+1, process[i]);
        if (alloc[i] != -1)
            printf("B%d\n", alloc[i]+1);
        else
            printf("Not Allocated\n");
    }
    return 0;
}
```

SAMPLE INPUT

Enter number of blocks: 5

Enter block sizes:

100 500 200 300 600

Enter number of processes: 4

Enter process sizes:

212 417 112 426

SAMPLE OUTPUT

Process Size Block

P1 212 B2

P2 417 B5

P3 112 B2

P4 426 Not Allocated

RESULT

Thus, memory was allocated using the First Fit method.

b) BEST FIT MEMORY ALLOCATION

AIM

To allocate memory using the Best Fit strategy.

ALGORITHM

1. Choose the smallest block that fits the process.
2. Allocate memory and update block size.
3. Display results.

SOURCE CODE

```
#include <stdio.h>
int main() {
    int nb, np, i, j;
    int block[10], process[10], alloc[10];
    printf("Enter number of blocks: ");
    scanf("%d", &nb);
    printf("Enter block sizes:\n");
    for (i = 0; i < nb; i++)
        scanf("%d", &block[i]);
    printf("Enter number of processes: ");
    scanf("%d", &np);
    printf("Enter process sizes:\n");
    for (i = 0; i < np; i++) {
        scanf("%d", &process[i]);
        alloc[i] = -1;
    }
    for (i = 0; i < np; i++) {
        int best = -1;
        for (j = 0; j < nb; j++) {
            if (block[j] >= process[i]) {
                if (best == -1 || block[j] < block[best])
                    best = j;
            }
        }
```

```
        if (best != -1) {
            alloc[i] = best;
            block[best] -= process[i];
        }
    }
    printf("\nProcess\tSize\tBlock\n");
    for (i = 0; i < np; i++) {
        printf("P%d\t%d\t", i+1, process[i]);
        if (alloc[i] != -1)
            printf("B%d\n", alloc[i]+1);
        else
            printf("Not Allocated\n");
    }
    return 0;
}
```

SAMPLE INPUT

Enter number of blocks: 5

Enter block sizes:

100 500 200 300 600

Enter number of processes: 4

Enter process sizes:

212 417 112 426

SAMPLE OUTPUT

Process Size Block

P1 212 B4

P2 417 B2

P3 112 B3

P4 426 B5

RESULT

Thus, memory was allocated using the Best Fit method.

c) WORST FIT MEMORY ALLOCATION

AIM

To allocate memory using the Worst Fit strategy.

ALGORITHM

1. Choose the largest available block.
2. Allocate memory.
3. Display results.

SOURCE CODE

```
#include <stdio.h>
int main() {
    int nb, np, i, j;
    int block[10], process[10], alloc[10];
    printf("Enter number of blocks: ");
    scanf("%d", &nb);
    printf("Enter block sizes:\n");
    for (i = 0; i < nb; i++)
        scanf("%d", &block[i]);
    printf("Enter number of processes: ");
    scanf("%d", &np);
    printf("Enter process sizes:\n");
    for (i = 0; i < np; i++) {
        scanf("%d", &process[i]);
        alloc[i] = -1;
    }
    for (i = 0; i < np; i++) {
        int worst = -1;
        for (j = 0; j < nb; j++) {
            if (block[j] >= process[i]) {
                if (worst == -1 || block[j] > block[worst])
                    worst = j;
            }
        }
```

```
        if (worst != -1) {
            alloc[i] = worst;
            block[worst] -= process[i];
        }
    }
    printf("\nProcess\tSize\tBlock\n");
    for (i = 0; i < np; i++) {
        printf("P%d\t%d\t", i+1, process[i]);
        if (alloc[i] != -1)
            printf("B%d\n", alloc[i]+1);
        else
            printf("Not Allocated\n");
    }
    return 0;
}
```

SAMPLE INPUT

Enter number of blocks: 5

Enter block sizes:

100 500 200 300 600

Enter number of processes: 4

Enter process sizes:

212 417 112 426

SAMPLE OUTPUT

Process Size Block

P1 212 B5

P2 417 B2

P3 112 B5

P4 426 Not Allocated

RESULT

Thus, memory was allocated using the Worst Fit method.

Ex: No: 10	Installation of Linux Guest Operating System using VMware
Date:	

AIM:

To install a Linux operating system as a guest OS using VMware virtualization software and to understand the concept of virtualization.

DESCRIPTION

Virtualization allows multiple operating systems to run on a single physical machine by creating virtual machines. VMware provides a platform to install and run guest operating systems such as Linux without affecting the host operating system. In this experiment, a Linux distribution is installed as a guest OS using VMware.

SYSTEM REQUIREMENTS

Hardware Requirements

- Intel / AMD processor with virtualization support
- Minimum 4 GB RAM
- Minimum 20 GB free hard disk space

Software Requirements

- **Host Operating System:** Windows / Linux
- **Virtualization Software:** VMware Workstation
- **Guest OS:** Ubuntu Linux ISO file

PROCEDURE

1. Install VMware Workstation on the host system.
2. Download a Linux ISO file (Ubuntu) from the official website.
3. Launch VMware Workstation and click Create a New Virtual Machine.
4. Select Typical (Recommended) and click Next.
5. Choose Installer disc image file (ISO) and browse the Ubuntu ISO file.
6. Select Linux as the guest operating system and choose Ubuntu as the version.
7. Specify the virtual machine name and storage location.
8. Allocate memory (RAM) and processor settings for the virtual machine.

9. Specify the disk size and choose Split virtual disk into multiple files.
10. Click **Finish** to create the virtual machine.
11. Power on the virtual machine and start the Ubuntu installation.
12. Follow the on-screen instructions to complete the Linux installation.
13. After installation, restart the virtual machine and log in to the Linux desktop.

SAMPLE OUTPUT

Ubuntu Linux desktop successfully loaded inside VMware virtual machine.

RESULT

Thus, a Linux guest operating system was successfully installed using VMware virtualization software.